
Design Document

for

FindIT

Version 1.0

Prepared by Team WebVortex

Group 2:

Rudra Sinha
Siddharth Miglani
Dharajiya Yug Harshadbhai
Pratul Koolwal
Rohan Gauranga Potukuchi
Nischay Agarwal
Vihaan Sapra
Suyash Kapoor
Aarush Narendra Ghatе
Shravan Agrawal

230880
231006
230362
230782
230864
230705
231149
231066
230017
230984

Group Name: WebVortex

rudrasinha23@iitk.ac.in
siddharthm23@iitk.ac.in
dharajiya23@iitk.ac.in
pratulk23@iitk.ac.in
rohangp23@iitk.ac.in
nischaya23@iitk.ac.in
svihaan23@iitk.ac.in
suyashk23@iitk.ac.in
aarushng23@iitk.ac.in
ashravan23@iitk.ac.in

Course: CS253

Mentor TA: Hemang Mohanlal Khatri

Date: 7 February 2025



CONTENTS.....	2
REVISIONS.....	3
1 CONTEXT DESIGN.....	4
1.1 CONTEXT MODEL.....	4
1.2 HUMAN INTERFACE DESIGN.....	5
2 ARCHITECTURE DESIGN.....	15
3 OBJECT-ORIENTED DESIGN.....	18
3.1 USE CASE DIAGRAM.....	18
3.2 CLASS DIAGRAM.....	21
3.3 SEQUENCE DIAGRAM	22
3.4 STATE DIAGRAM	31
4 PROJECT PLAN.....	32
APPENDIX A - GROUP LOG.....	34

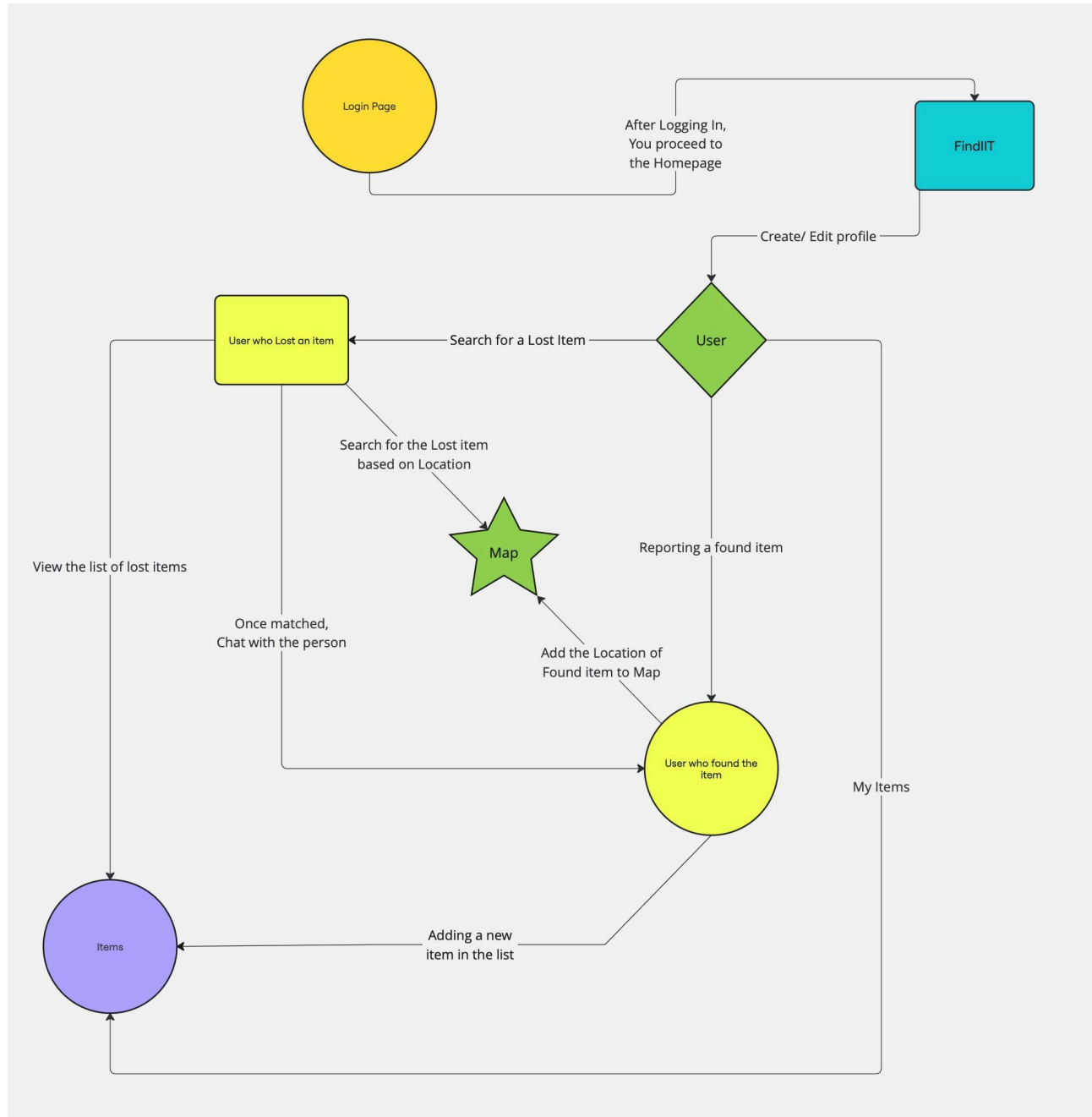
Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	Rudra Sinha Siddharth Miglani Dharajiya Yug Harshadbhai Pratul Koolwal Rohan Gauranga Potukuchi Nischay Agarwal Vihaan Sapra Suyash Kapoor Aarush Narendra Ghate Shravan Agarwal	First Draft	07/02/25

1 Context Design

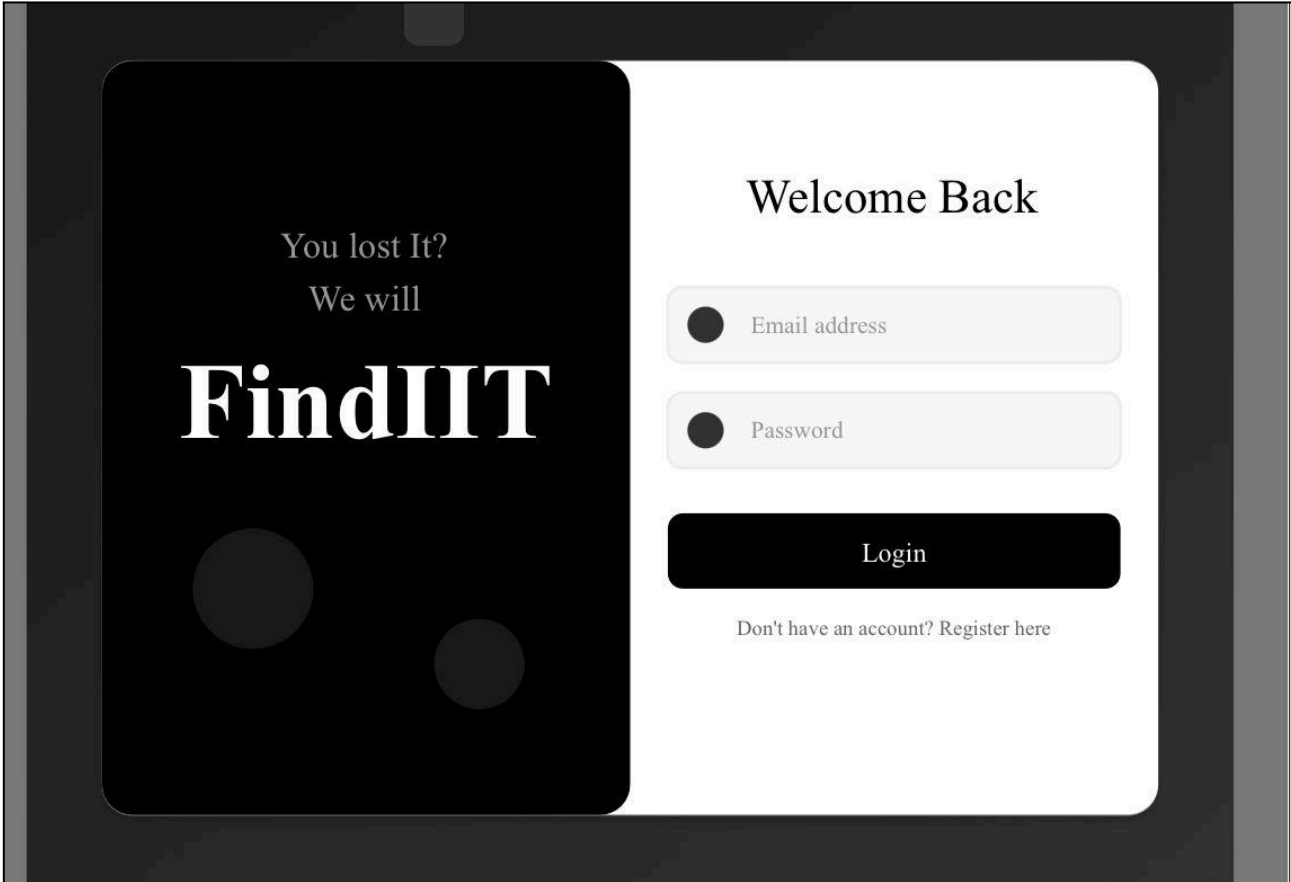
1.1 Context Model

The context model shows the various aspects of the system which interact with each other. The entities are Users who found the item and Users who lost the item who interact with the Map and Chat feature. The database of FindIIT stores all the data of the system. There will be an OTP API which sends unique codes for login and registration purposes.



1.2 Human Interface Design

Login Page
verify_credentials(email, password) <i>User can login via email and password</i>



The mockup shows a dark-themed login interface. On the left, a dark gray rounded rectangle contains the text "You lost It? We will" in a light gray serif font, followed by the "FindIIT" logo in a large, bold, white serif font. Below the logo are two faint, overlapping circles. On the right, a white rounded rectangle is set against the dark background. It features the heading "Welcome Back" in a black serif font. Below this are two light gray input fields with rounded corners; the first is labeled "Email address" and the second "Password", both in a light gray sans-serif font. Each input field has a small dark gray circle on its left side. Below the input fields is a solid black button with the word "Login" in a white sans-serif font. At the bottom of the white area is a link in a small, light gray sans-serif font that reads "Don't have an account? Register here".

Registration page

new_user = register(name, email, password)
User can register via name, email and password

Join Us

Be a part of

FindIIT



Create an Account

Register

Already have an account? **Login here**

OTP Verification page**verify_otp(user, otp)***User confirms email via otp sent to email*

OTP Verification

Enter the OTP sent to your email

Verify

Didn't receive the code? **Resend OTP**

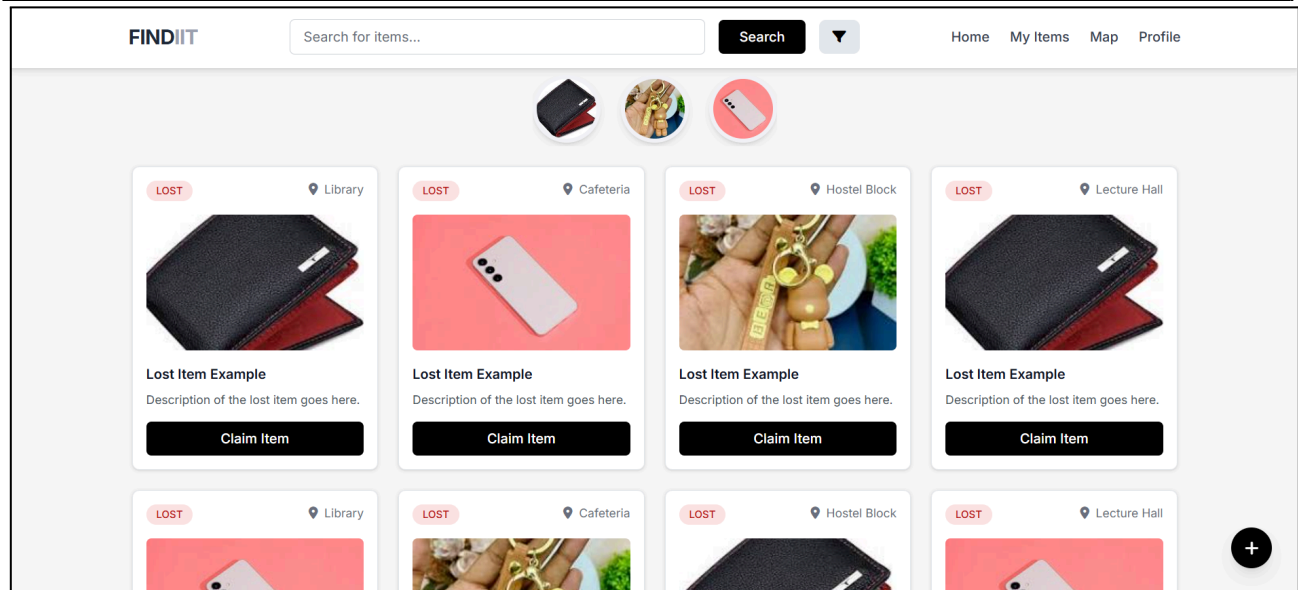
Home page

search(item_name)

User can search all items via name, tags, content in description

filter(status, location_area, tag, date, time)

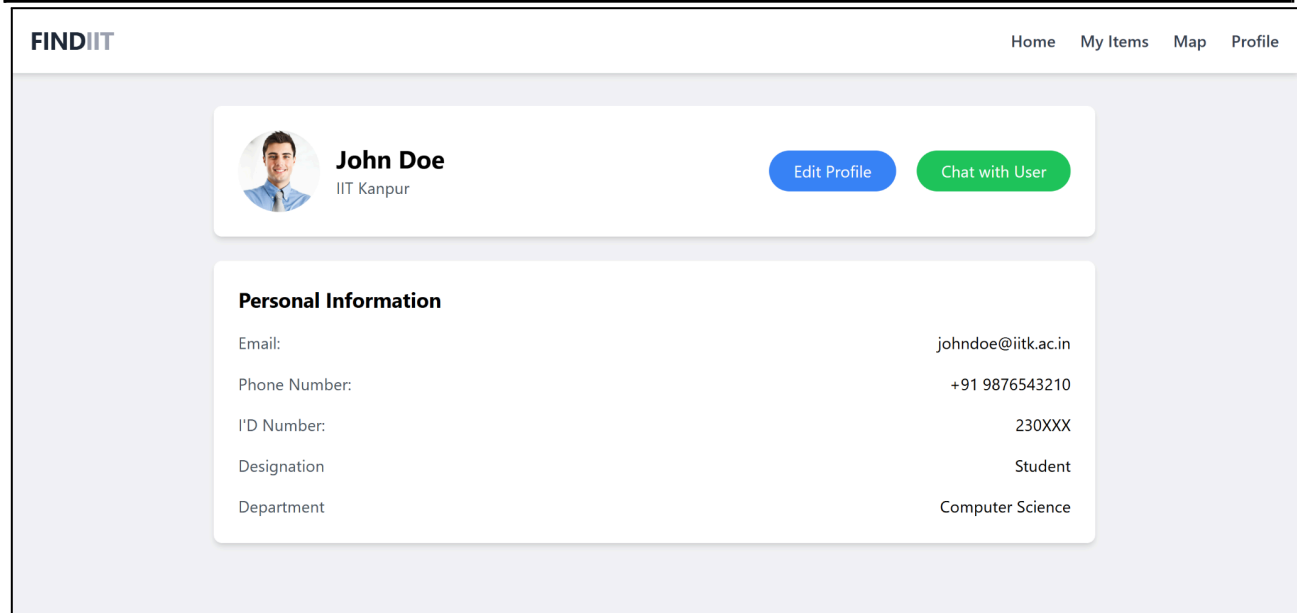
User can filter items via status(lost/found), date and time posted, tags



User page

view_user(user)

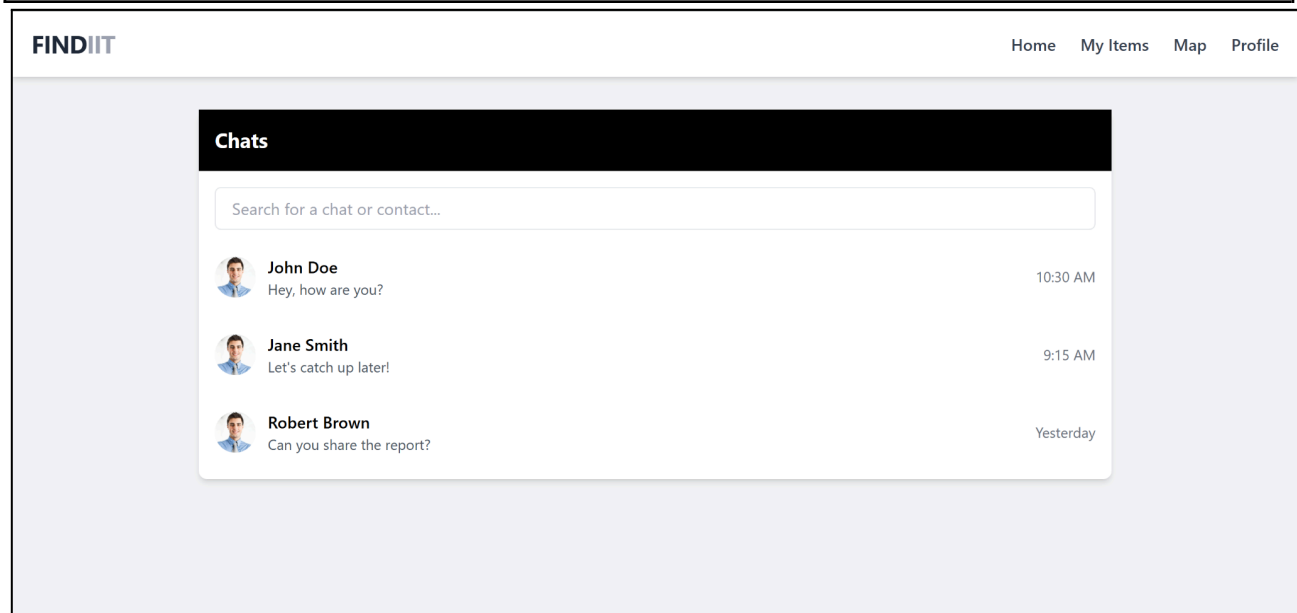
User can view user's details like name, number, email, roll number, department and designation

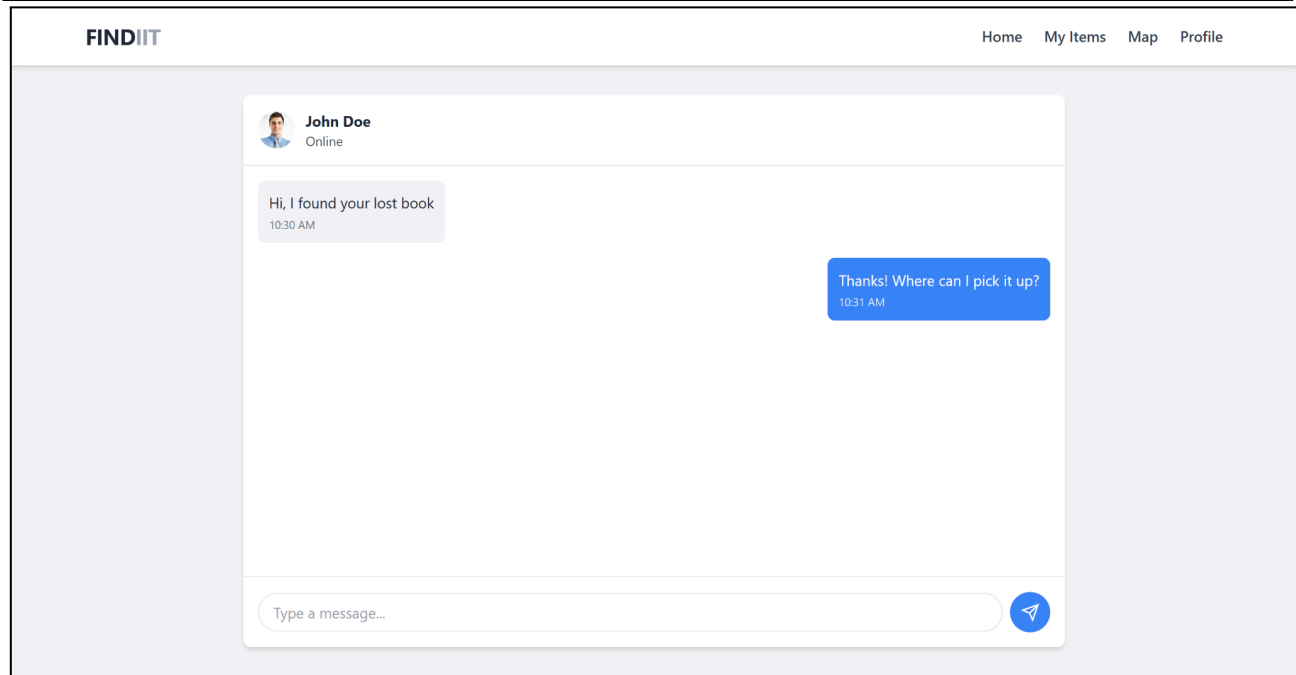


All chats page

view_users_chats(user)

User can view users he has chat history with



Chat page**send_msg(user, text)***User can send message to other user***view_msgs(user)***User can view chat history with another user*

Edit profile page

edit_profile(name, number, email, department, designation)
Users can edit their name, number, email, department, designation.

FINDIITHomeMy ItemsMapProfile

Edit Profile



Profile Picture
[Choose File](#) No file chosen

Full Name

John Doe

Email

johndoe@iitk.ac.in

Phone Number

+91 9876543210

ID Number

230XXX

Designation

Student

Department

Computer Science

Cancel

Save Changes

Posting page

new_item = report(status, item_name, description, tags, location, number, image)
User can post new item with details like status(lost/found), name, tags, description, location, number, image

FINDIIT[Home](#)[My Items](#)[Map](#)[Profile](#)

Report an Item

Item Status
☐ Lost Item ☐ Found Item

Select Item Category
Wallet

Item Description
Write a brief description of the item

Add tags (separate tags by commas)
e.g. Metal, Black, Wallet

Location or Gate


Latitude, Longitude

Contact Details (Point of Contact)
Phone number, email, etc.

Upload Image of the Item
 No file chosen

Item page

view_item(item)

Users can view item details like name, description, tags, location, status(lost/found), date and time posted, user that posted it.

FINDIITHomeMy ItemsMapProfile

Lost Item Details



Location



Latitude: 28.5456, Longitude: 77.2731

Status: Lost

Item Information

Category
Wallet

Description
Black leather wallet with metal trim

Tags
Metal Black Wallet

Contact Information

Posted By
John Doe

Contact Details
+91 98765 43210
john.doe@example.com

Time Information

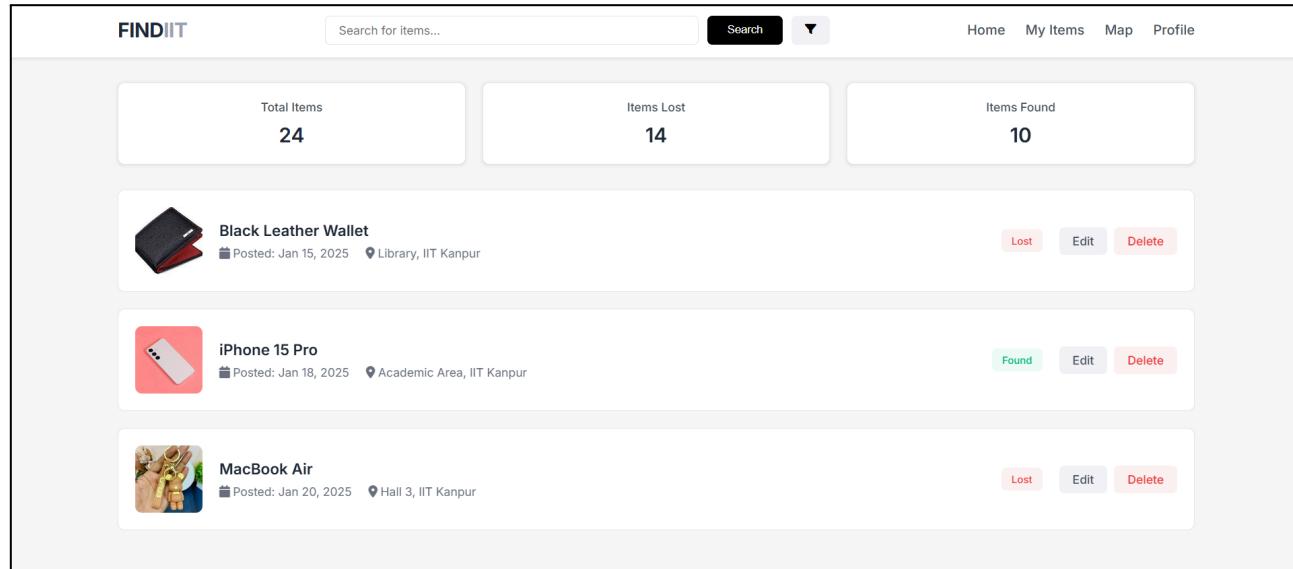
Lost Date
February 7, 2025

Posted Date
February 7, 2025

Contact

Items of User page

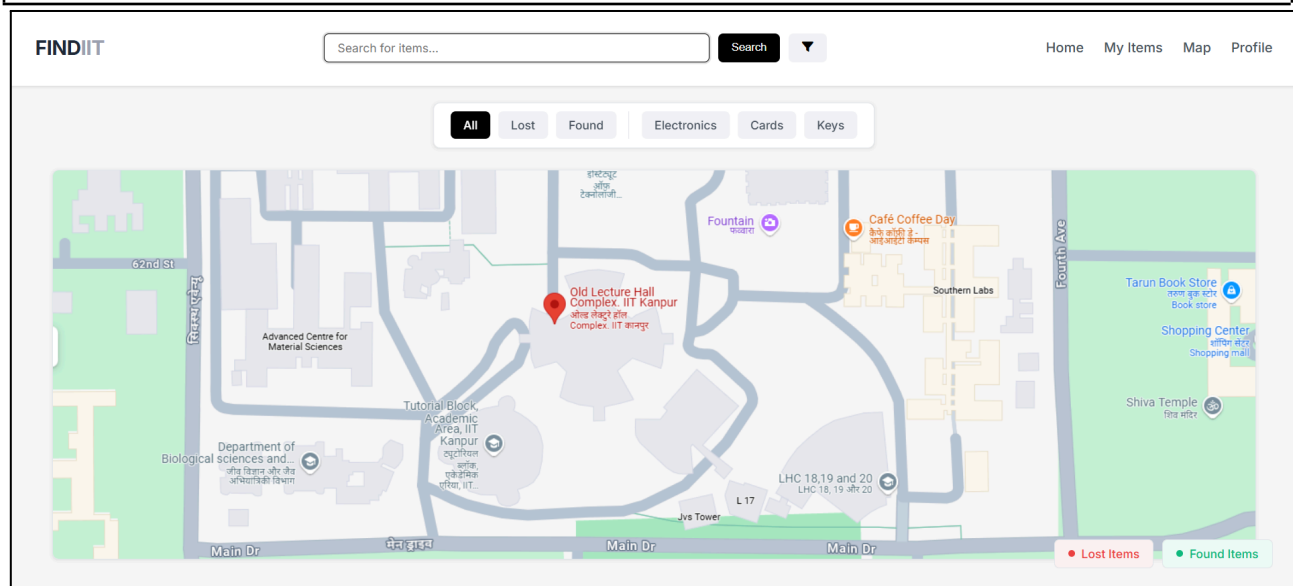
view_item_user(item, user)
Users can view items posted by a particular user



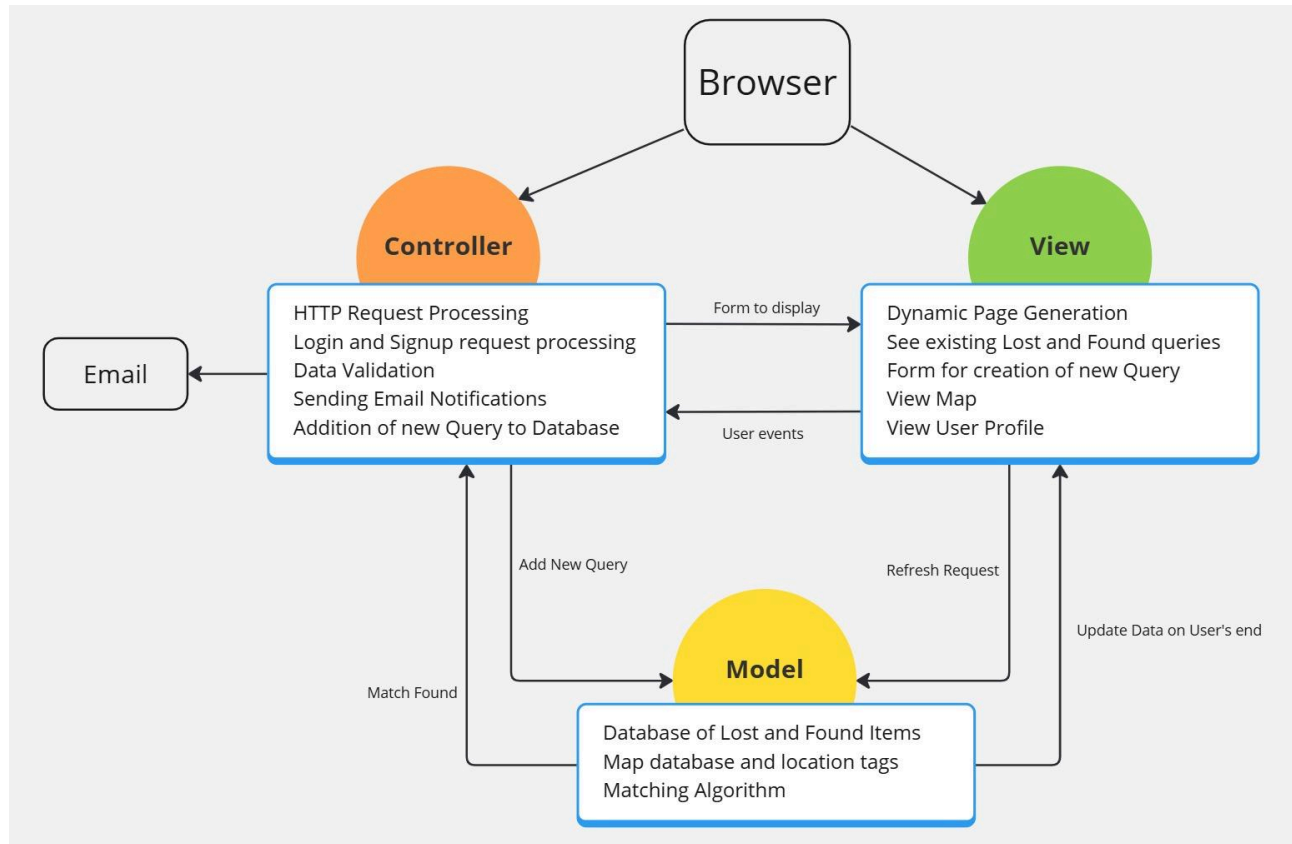
Map page

search_item(location_area, status, tags)
Users can filter items based on location_area, status(lost/found), tags

view_items(location_area)
Users can view items inside a location_area

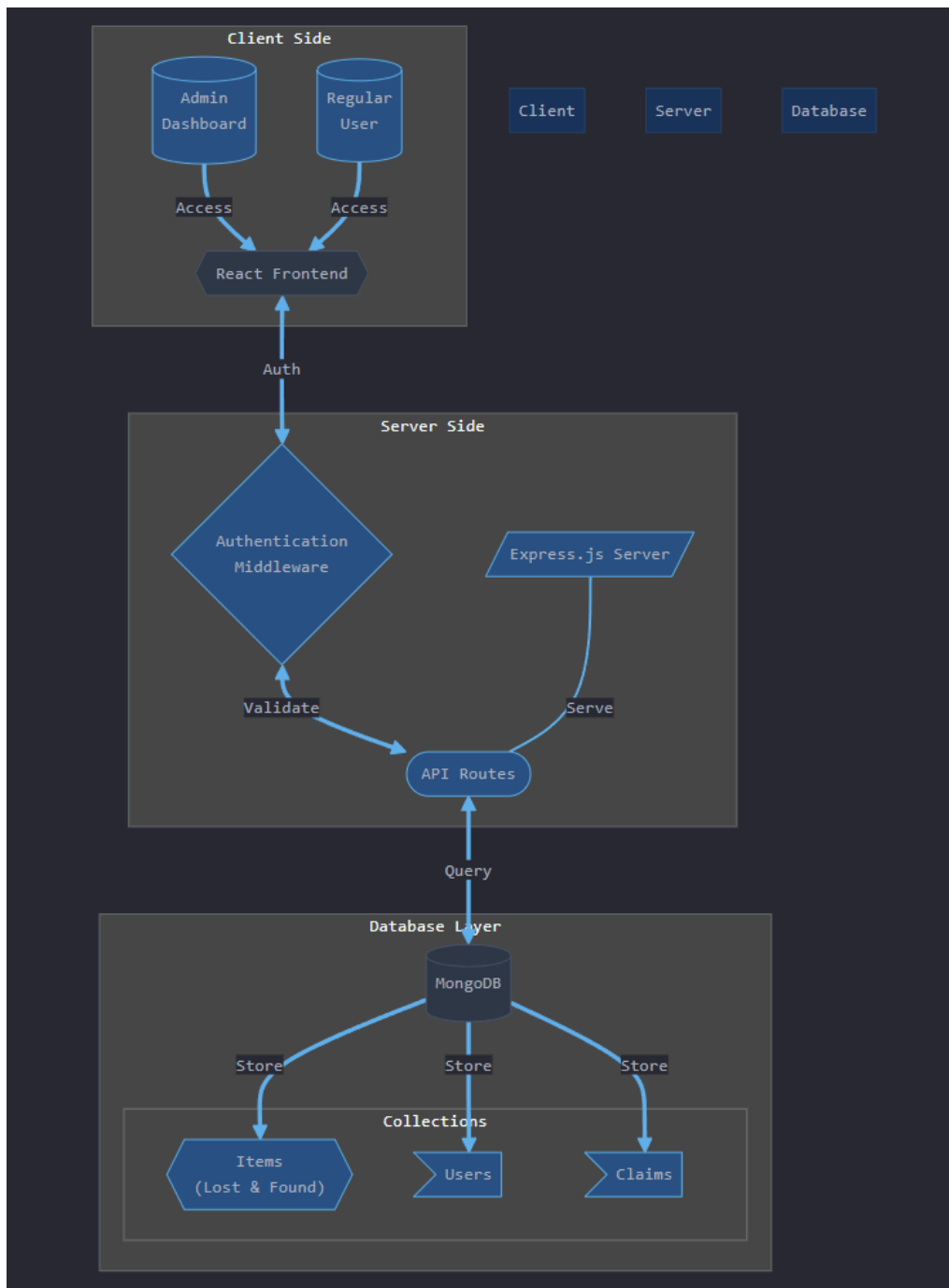


2 Architecture Design



This software is based on **Model-View-Controller** architecture. The main reasons behind this choice are:-

1. There are multiple ways to interact with data. Users can view all queries, filter them, can view only some of them or can see all queries on a map. This architecture allows the same data to be presented in such varied formats.
2. This architecture allows new queries to be added without directly affecting other users.
3. Adds a layer of security between user and database to ensure privacy and security of data.

Tech Stack Architecture:

This architecture should meet the following non-functional requirements:

- **Scalability:**

The architecture is designed to be highly scalable. Since we are using the MERN stack (MongoDB, Express.js, React.js, and Node.js), we can efficiently handle an increasing number of users and data. The backend (Node.js with Express.js) and the database (MongoDB) can be scaled horizontally by deploying multiple instances behind a load balancer. Additionally, MongoDB's sharding mechanism allows distributing data across multiple servers, ensuring efficient performance at scale. Cloud-based services like AWS, Firebase, or DigitalOcean can be leveraged to further enhance scalability while maintaining user privacy.

- **Performance:**

The MERN stack provides a lightweight, asynchronous, and event-driven architecture, which ensures high performance and responsiveness. The use of RESTful APIs for communication between the frontend (React.js) and backend (Express.js) minimizes latency. MongoDB's document-oriented NoSQL structure enables fast data retrieval, while caching mechanisms like Redis can be integrated to optimize query performance.

- **Security & Privacy:**

Security is a top priority, as the platform handles user-generated data, including personal details. For this reason, our architecture implements security measures such as the protection of sensitive data and enforcing access control. This is done by devising an authentication system which verifies the IITK email id of the user and prevents unauthorized access, all of which is implemented using the backend framework.

- **Interoperability:**

The MERN stack ensures compatibility across different platforms and devices. React.js provides a responsive UI that works seamlessly across web and mobile browsers. The backend's RESTful API allows integration with third-party services if required in the future. Additionally, the use of JavaScript throughout the stack (frontend, backend, and database interactions) ensures a uniform development experience.

- **Reliability:**

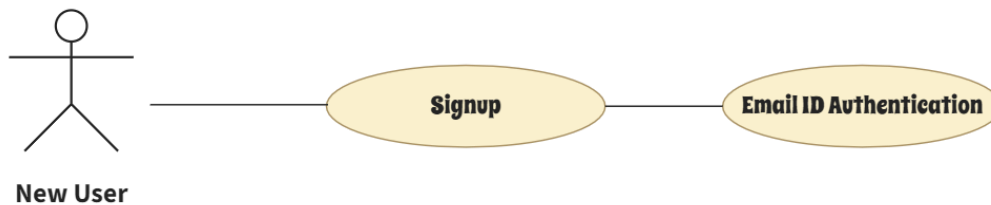
Through rigorous testing and beta testing, as well as a robust design achieved by using enhanced and flexible code quality, we plan to provide continuous integration and deployment within our architecture so that any bugs or glitches are prevented from occurring and the software works smoothly. We also plan to cover all details through thorough and up-to-date documentation, which will make it easy to use and convenient for users, while being accessible most of the time and dynamically updated as well.

3 Object Oriented Design

3.1 Use Case Diagrams

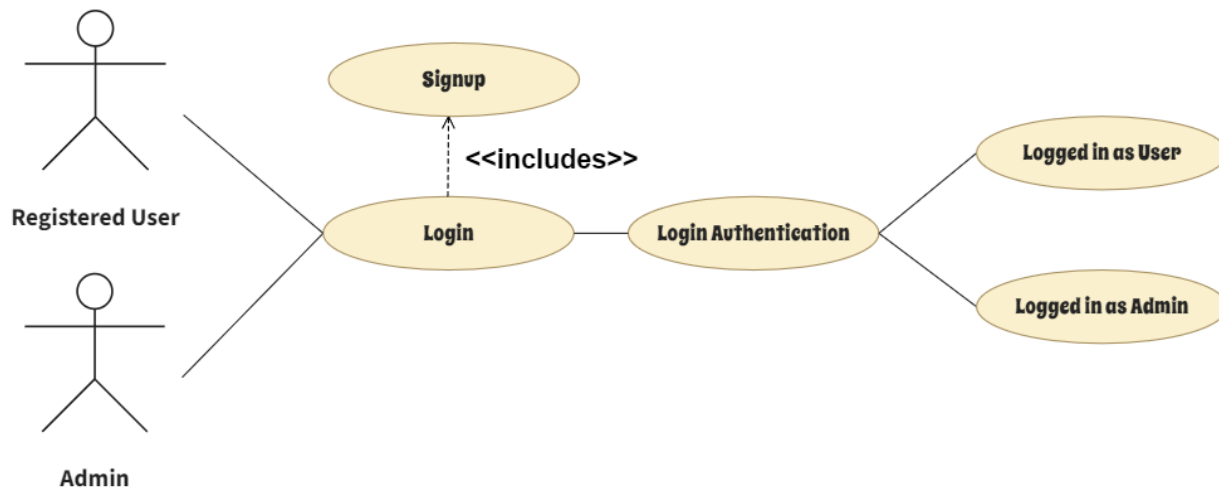
3.1.1 Use Case #1: User Registration

Purpose: New User Registration for students and Registered User Login



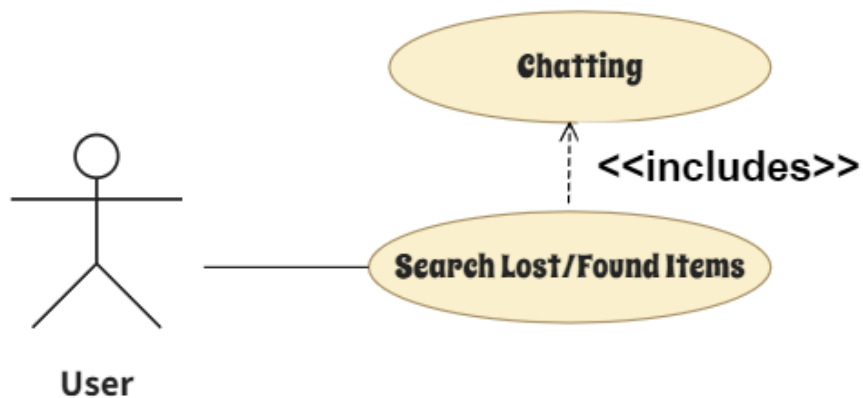
3.1.2 Use Case#2 : User Login

Purpose: Registered User Login



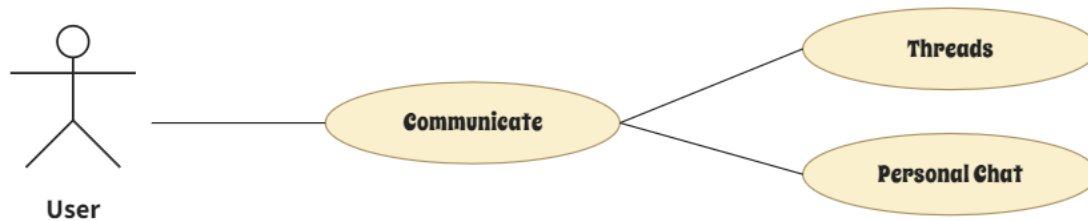
3.1.3 Use Case #3: Viewing Lost and Found items

Purpose: Searching through items to find a lost item or match with other found item



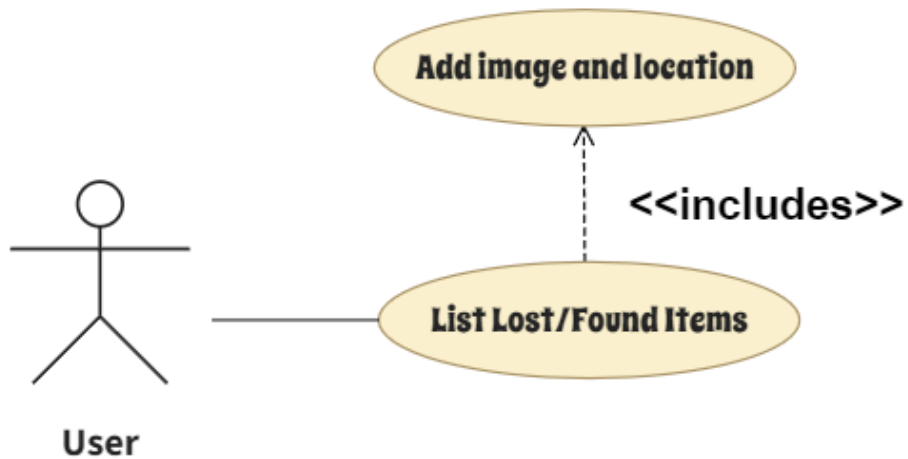
3.1.4 Use Case #4: Communicate

Purpose: To Chat and Confirm the Lost/Found Item



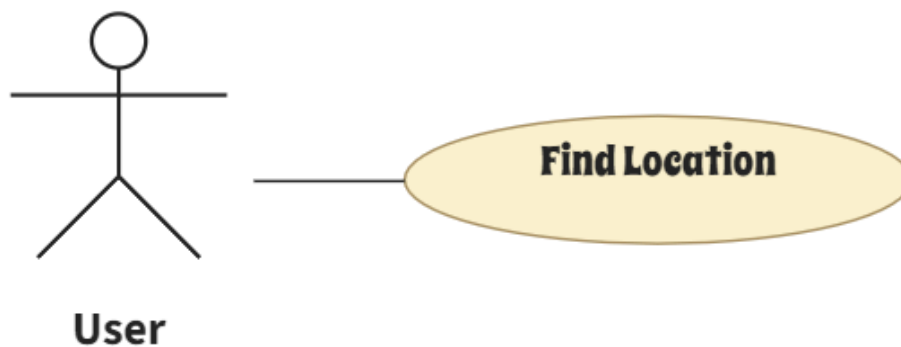
3.1.5 Use Case #5: Listing Lost /Found items

Purpose: Listing a lost/found item on the site



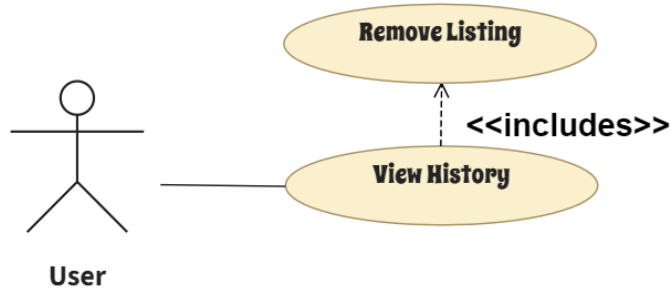
3.1.6 Use Case #6: Find Location

Purpose: To check location of lost/found items

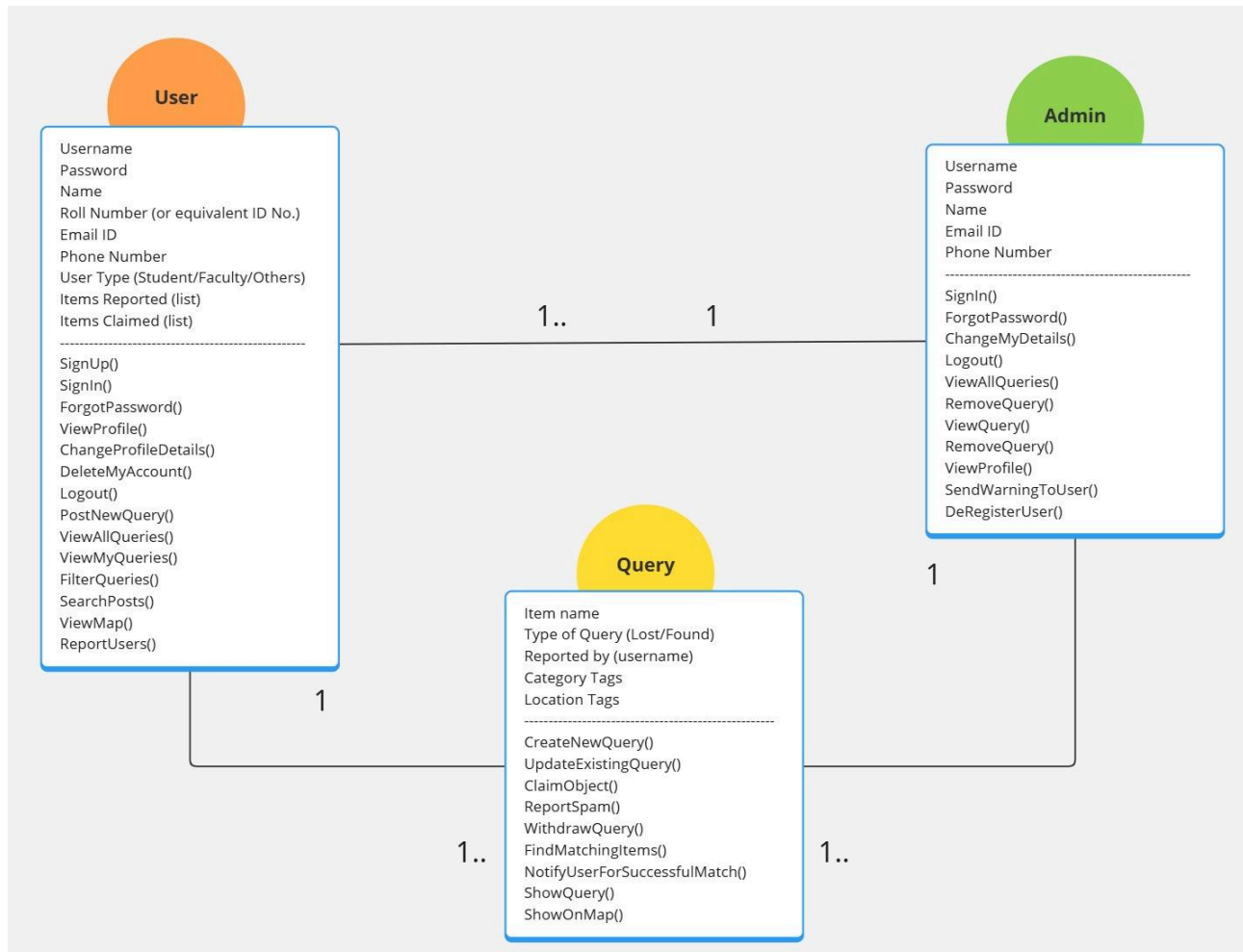


3.1.7 Use Case #7: View History

Purpose: To view previous listings made by the user

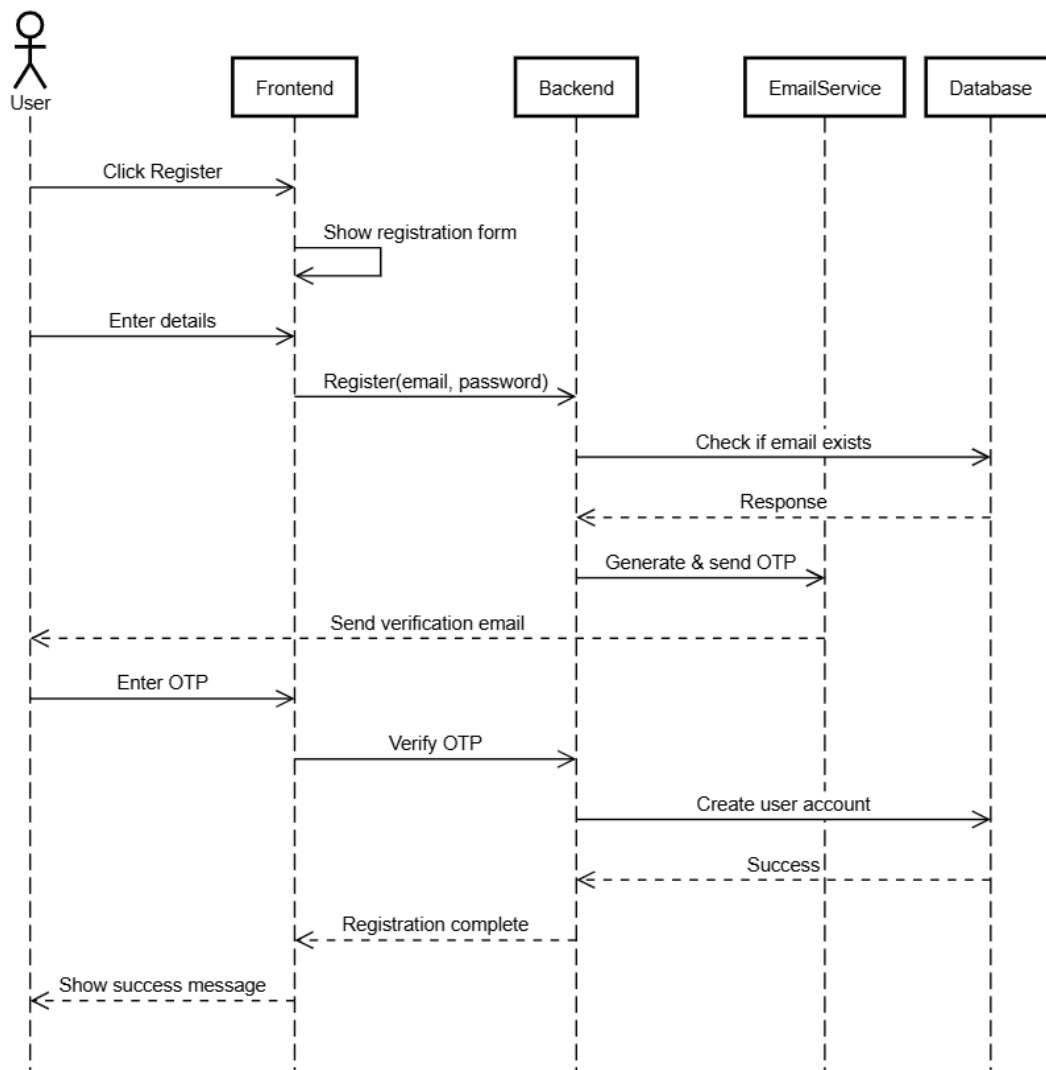


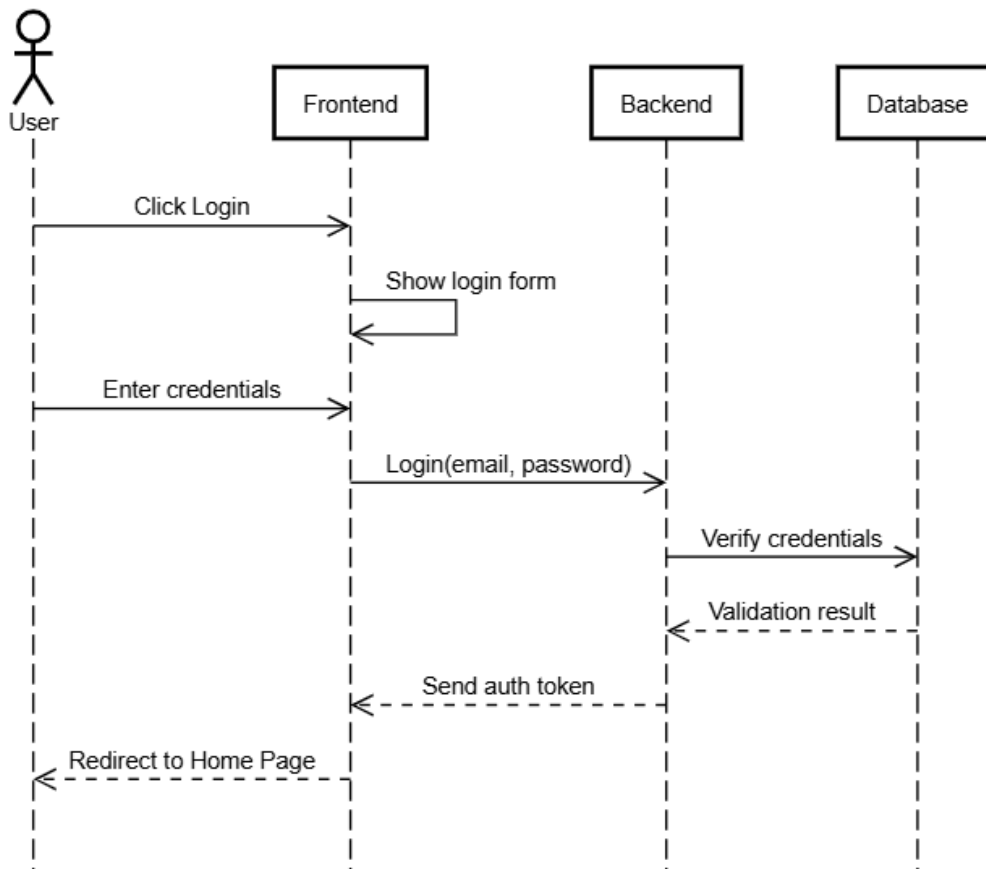
3.2 Class Diagrams



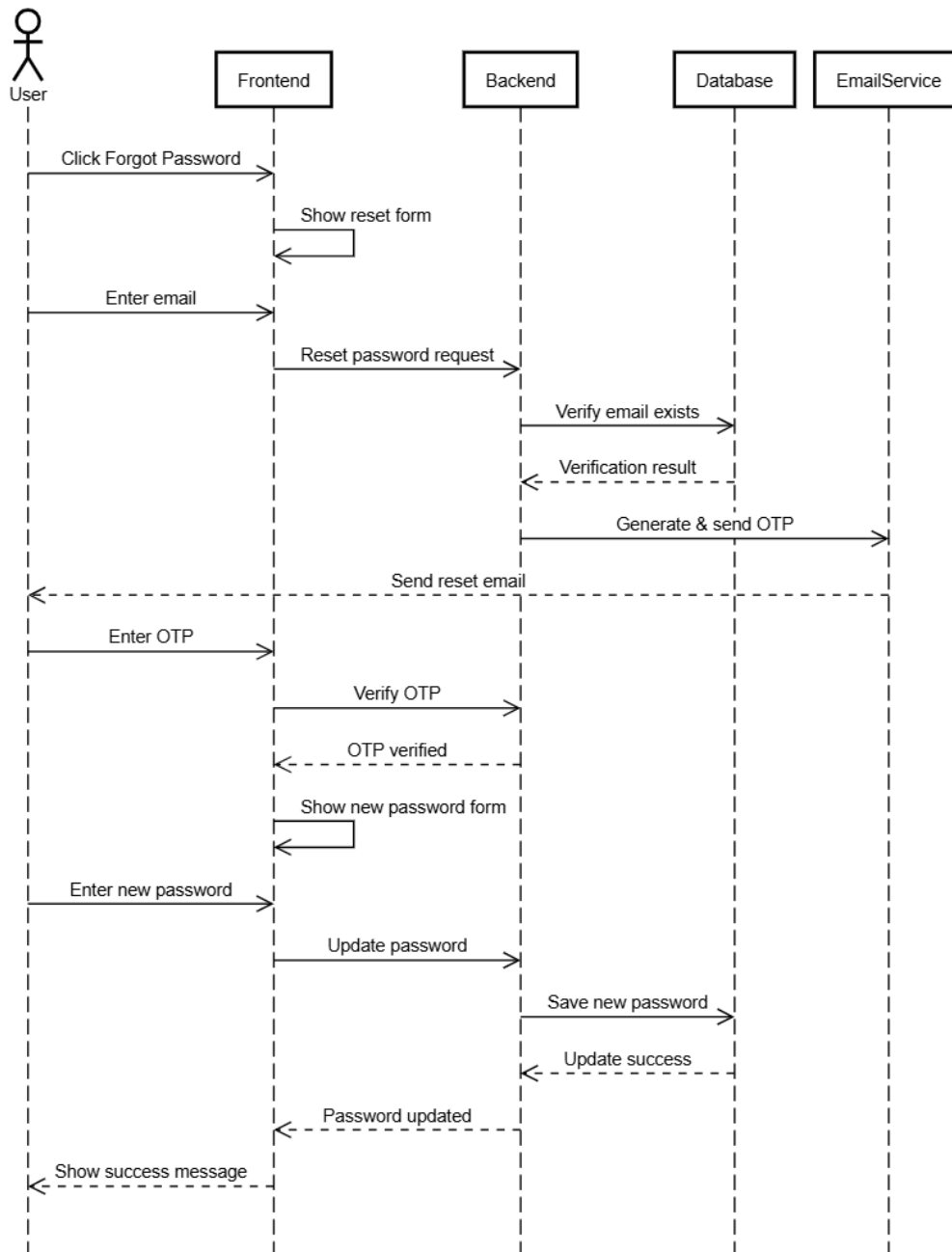
3.3 Sequence Diagrams

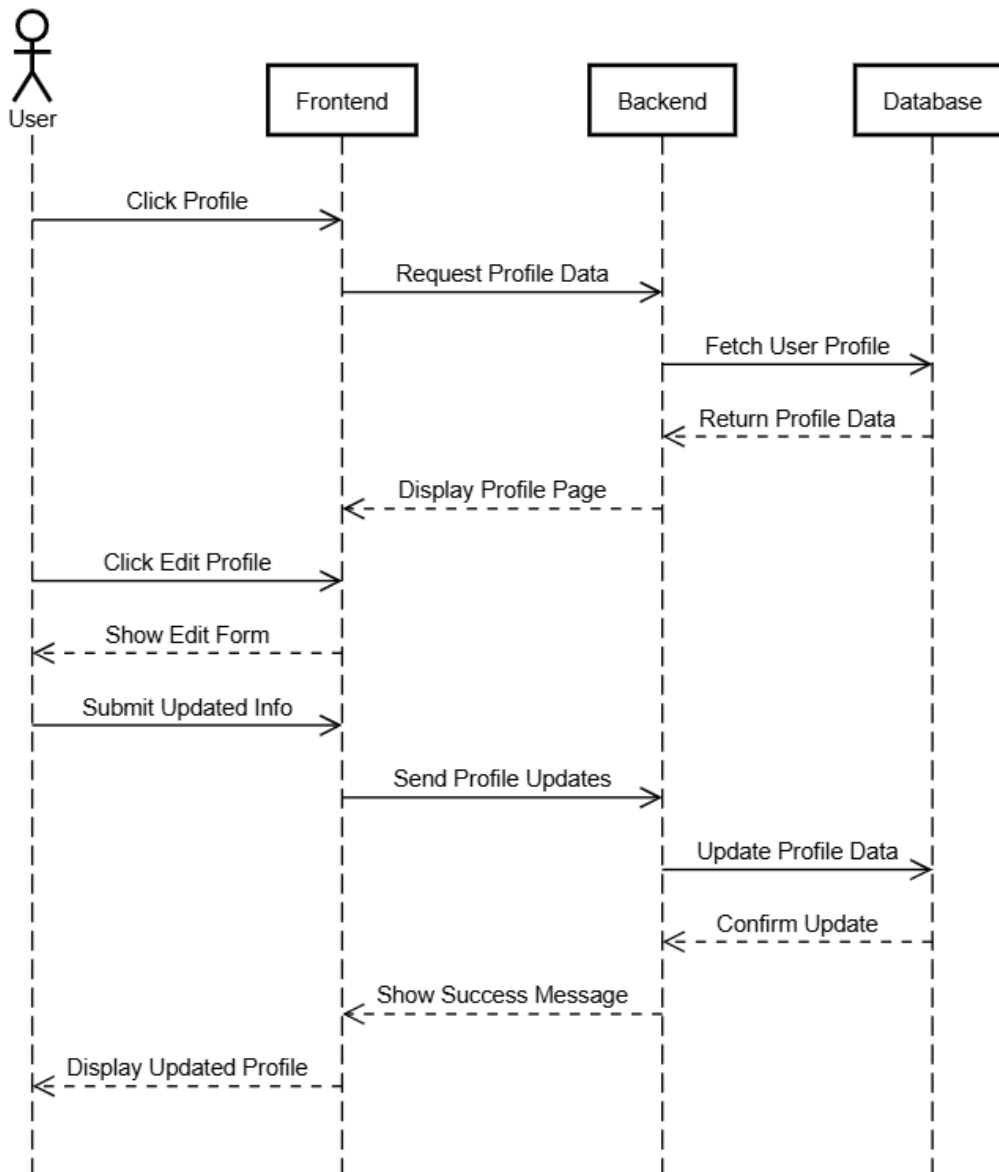
3.3.1 Register:

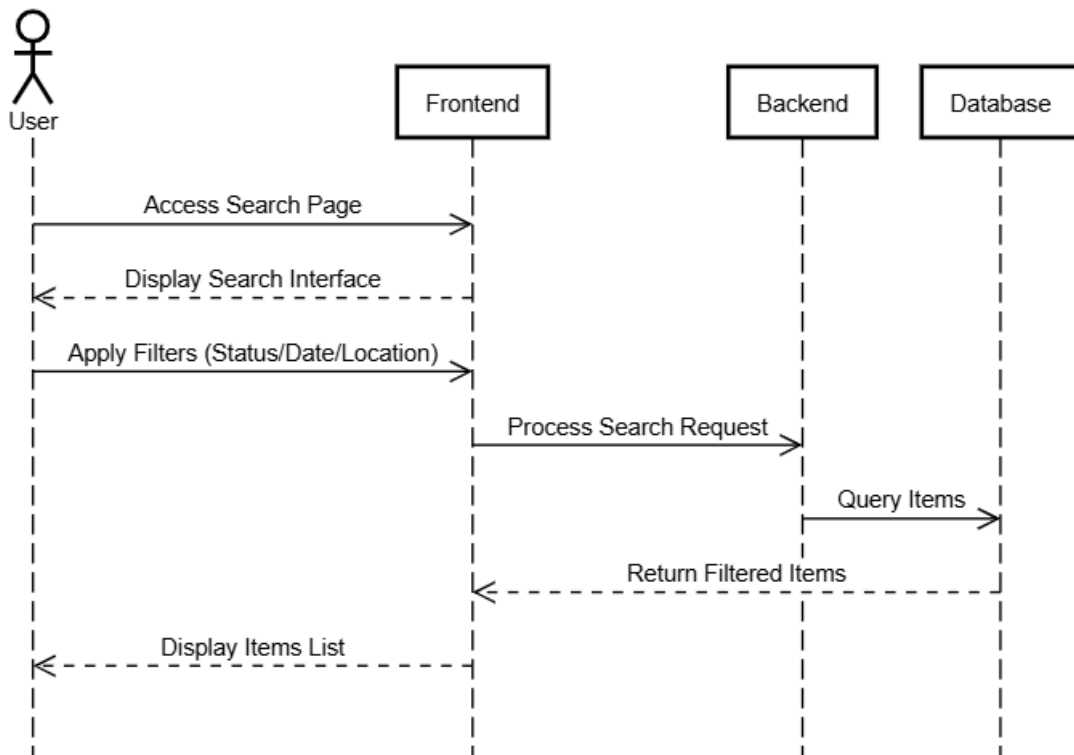


3.3.2 Login:

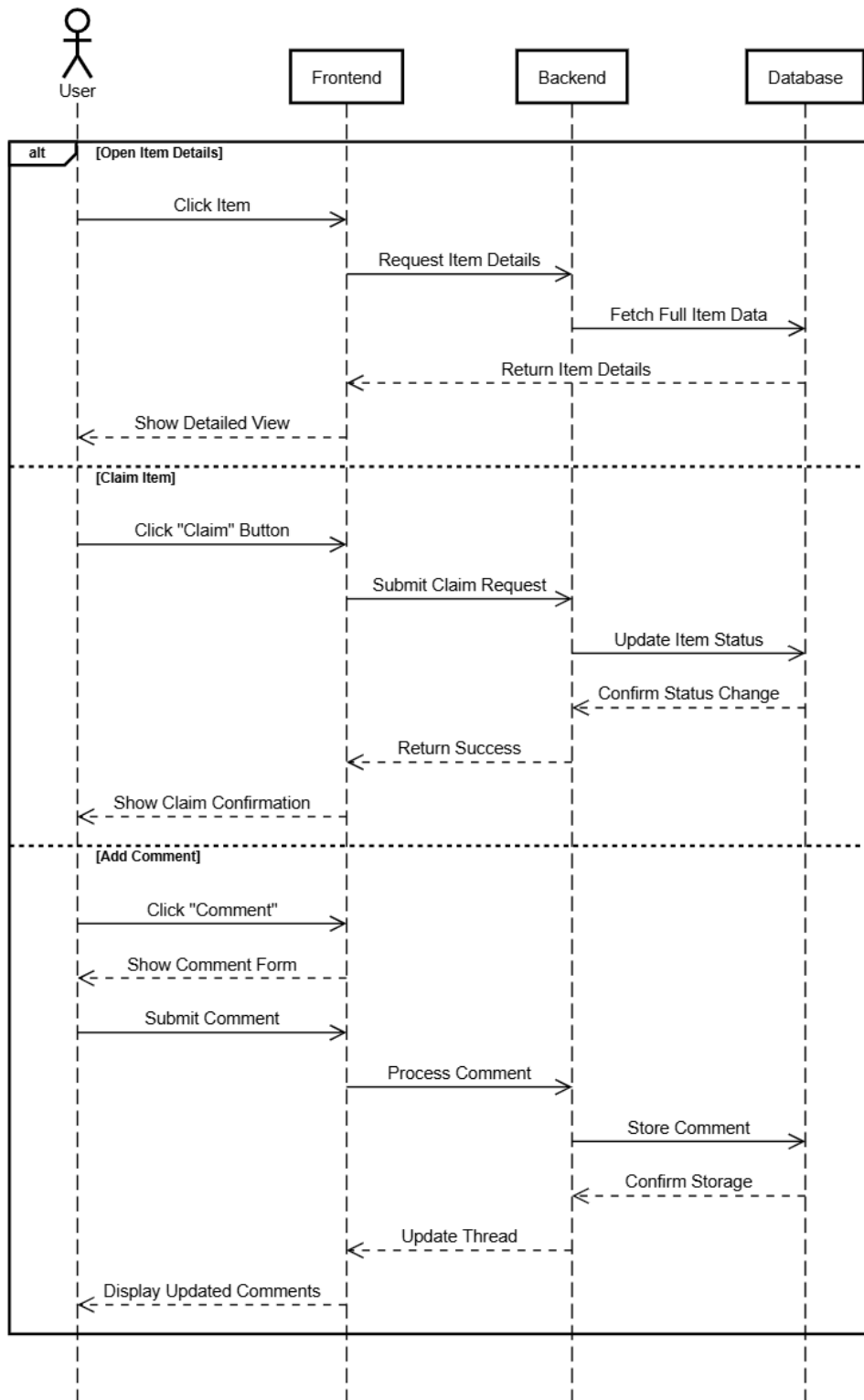
3.3.3 Forgot Password:

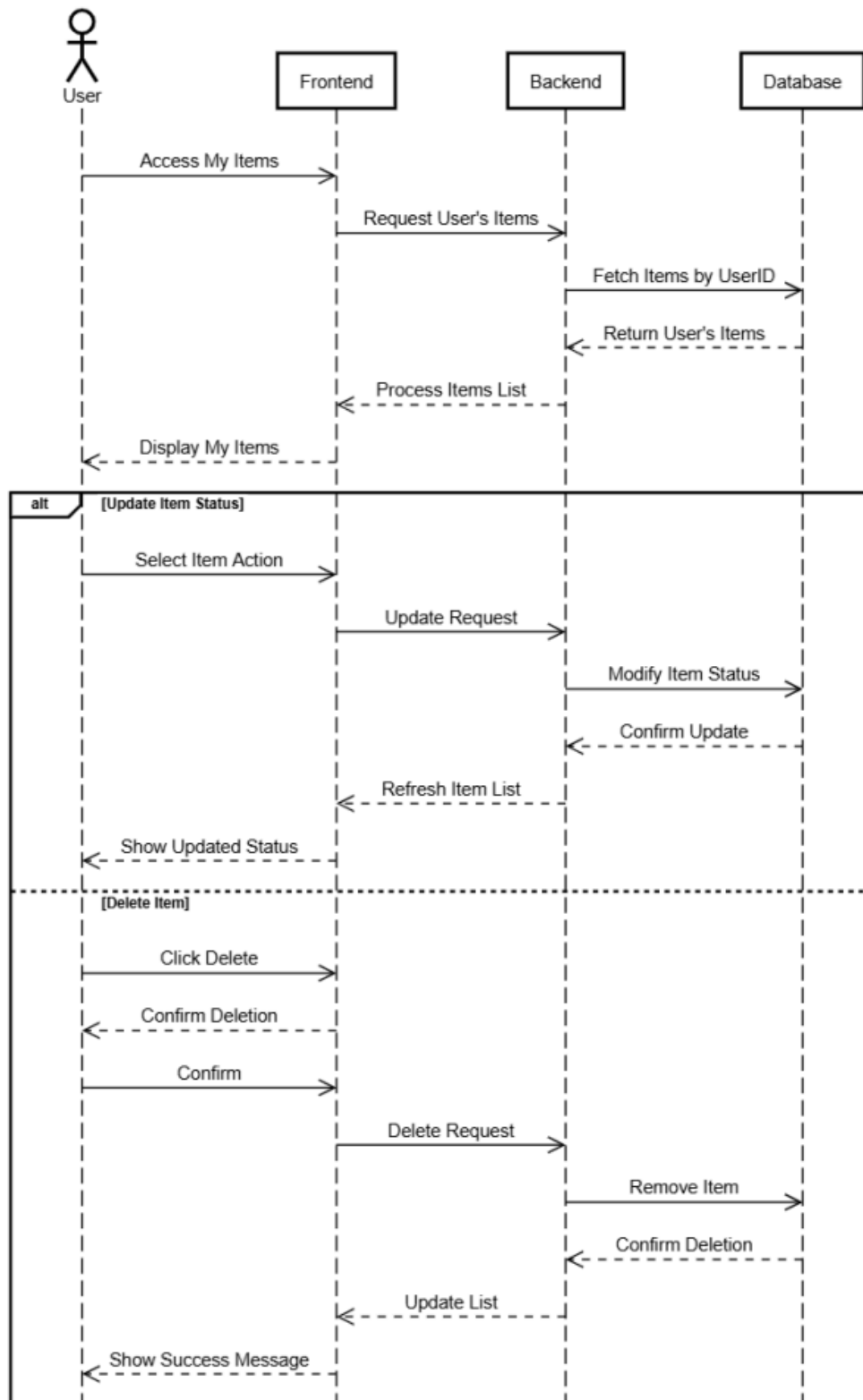


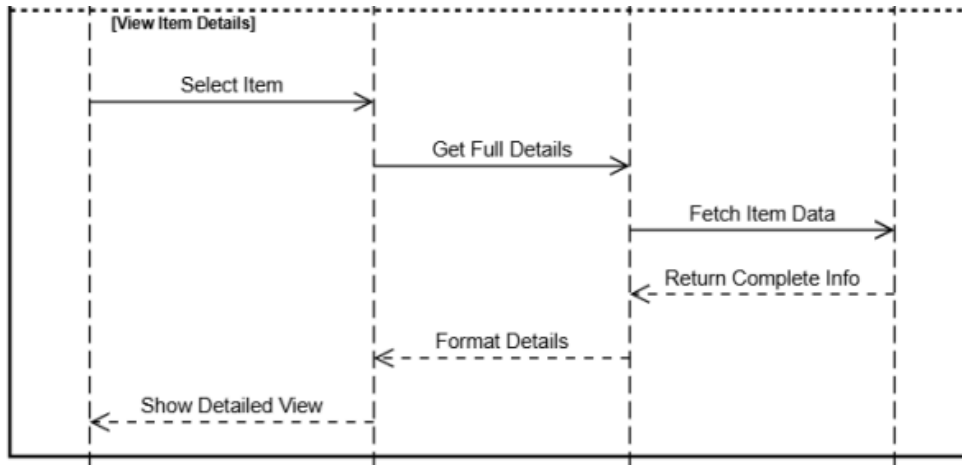
3.3.4 Edit Profile:

3.3.5 Search:

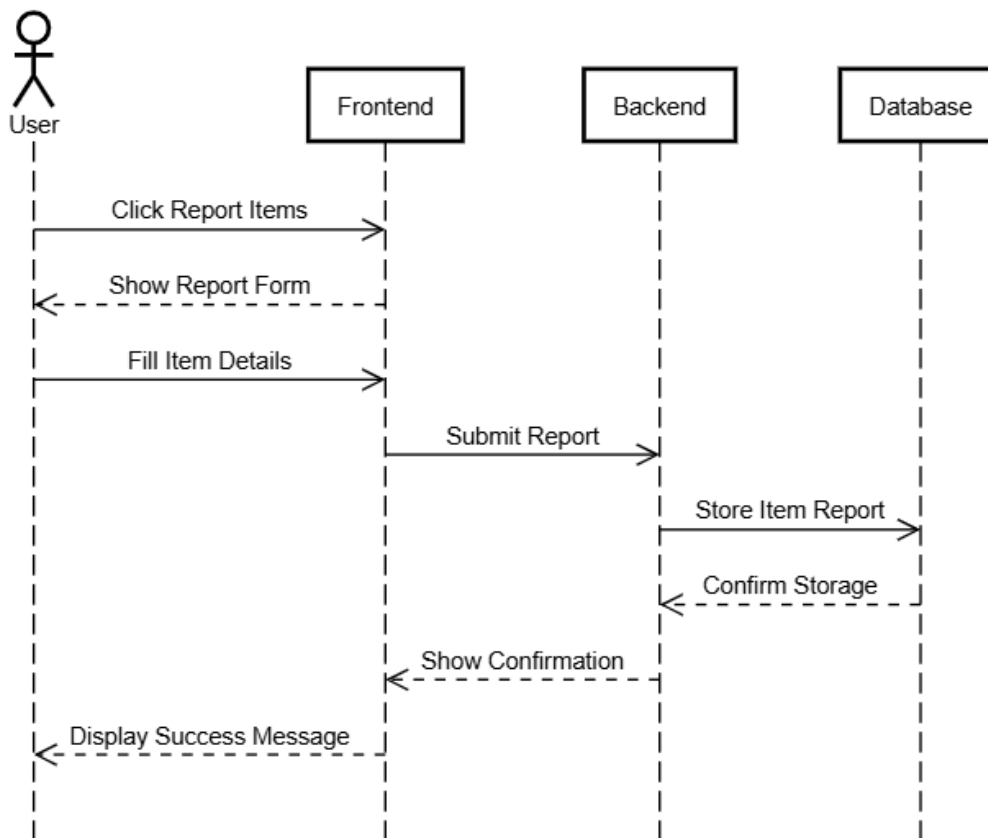
3.3.6 Item Page:

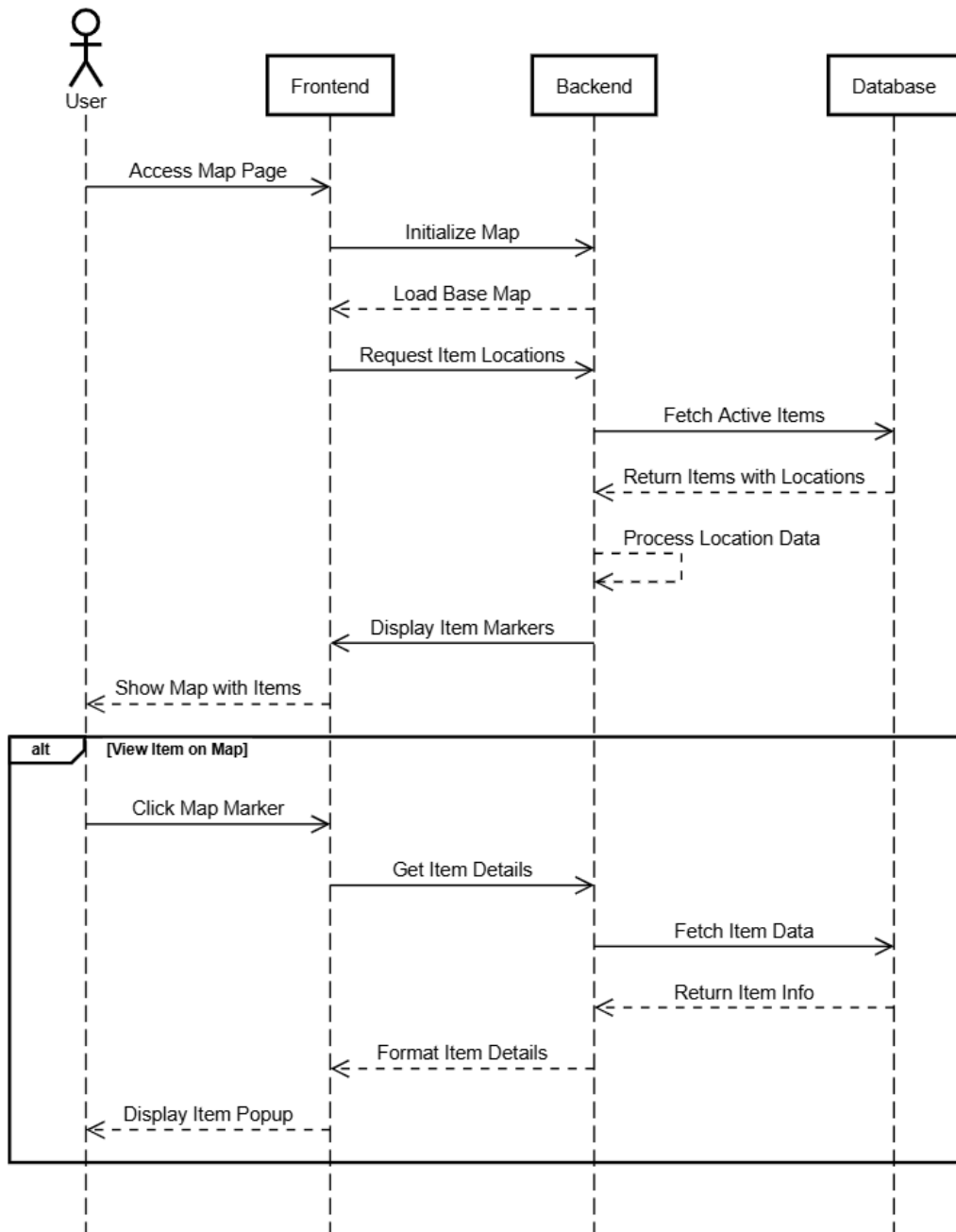


3.3.7 My Items:

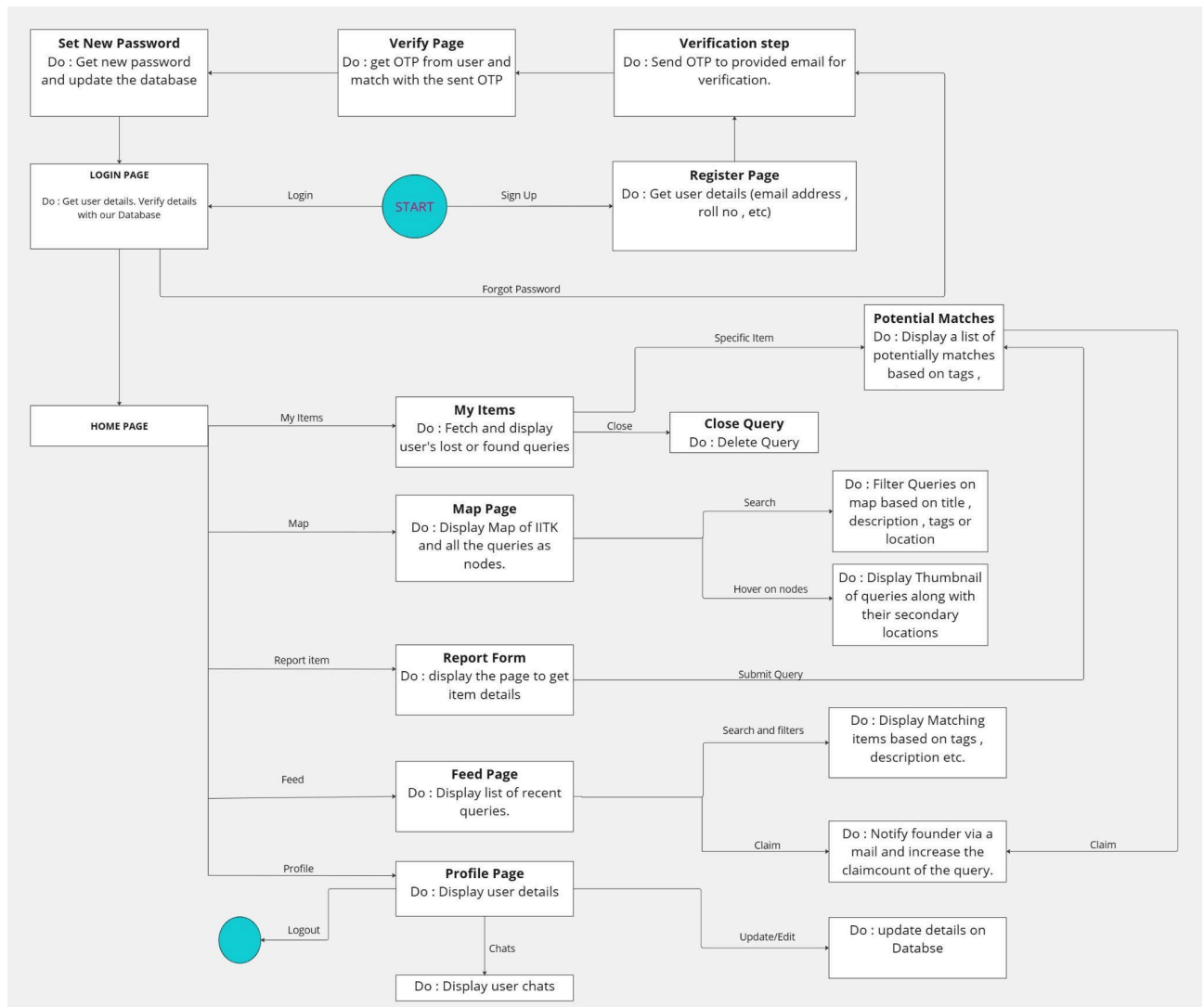


3.3.8 Report Item:

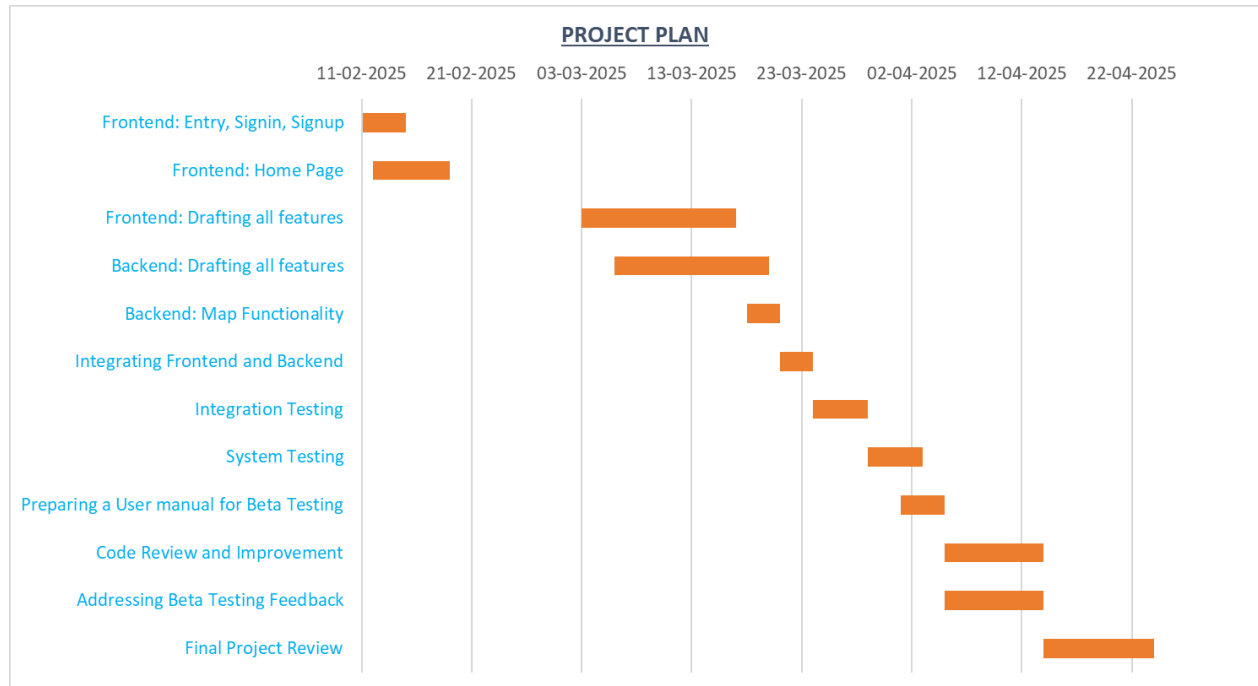


3.3.9 Map:

3.4 State Diagrams



4 Project Plan



<u>START DATE</u>	<u>END DATE</u>	<u>TASK DESCRIPTION</u>	<u>DURATION (days)</u>	<u>TEAM MEMBERS</u>
11-02-25	14-03-25	Frontend: Entry, Sign in, Signup	4	Suyash, Rudra
12-02-25	18-03-25	Frontend: Home Page	7	Shravan, Nischay, Vihaan
03-03-25	16-03-25	Frontend: Drafting all features	14	Rohan, Aarush, Pratul, Siddharth, Yug
06-03-25	19-03-25	Backend: Drafting all features	14	Vihaan, Shravan, Pratul, Siddharth, Yug, Rudra, Suyash, Nischay
18-03-25	20-03-25	Backend: Map Functionality	3	Rohan, Aarush
21-03-25	23-03-25	Integrating Frontend and Backend	3	Full Team
24-03-25	28-03-25	Integration Testing	5	Full Team
29-03-25	02-04-25	System Testing	5	Full Team

01-04-25	04-04-25	Preparing User manual for Beta Testing	4	Full Team
05-04-25	13-04-25	Code Review and Improvement	9	Full Team
05-04-25	13-04-25	Addressing Beta Testing Feedback	9	Full Team
14-04-25	23-04-25	Final Project Review	10	Full Team

The following are the broad classification of the tasks:

- 1. Front End Design:** A front-end designer not only creates the aesthetics and wireframes for the product but also turns them into reality via code. Any component of the website which the end User is able to manipulate can be classified as Front End.
- 2. Back End Development:** It is everything that the users don't see and contains behind-the-scenes activities that occur when performing any action on a website. It focuses primarily on databases, backend logic, APIs, and Servers.

Appendix A - Group Log

- We have made a WhatsApp group for effective communication amongst ourselves.
- We have created a private repository on GitHub on which we would be regularly pushing our codes to maintain efficiency and collaboration.
- We have broadly divided the team into parts, all members would be working on different aspects of frontend and backend so that all members understand the mechanism of software development and are able to implement it.

The following is the group log for the development of this SDD (Software Design Document).

Date	Meet type	Final outcome of meet
28 January 2025	Online	Forming groups among our team to tackle different aspects of the project
31 January 2025	Offline	Dividing sections of the Software Design Document, and assigning them to members of the team
2 February 2025	Offline	To check on the progress of all team members and clarification of specific aspects of software.
6 February 2025	Online	Finalisation of Project timeline and other aspects.
7 February 2025	Offline	Final meet amongst the team members as a robust draft of the Software Design Document.